

Design of a LISP-Based Microprocessor

Guy Lewis Steele Jr. and Gerald Jay Sussman

**Artificial Intelligence Laboratory
Massachusetts Institute of Technology**

Abstract:

We present a design for a class of computers whose "instruction sets" are based on LISP. LISP, like traditional stored-program machine languages and unlike most high-level languages, conceptually stores programs and data in the same way and explicitly allows programs to be manipulated as data. LISP is therefore a suitable language around which to design a stored-program computer architecture. LISP differs from traditional machine languages in that the program/data storage is conceptually an unordered set of linked record structures of various sizes, rather than an ordered, indexable vector of integers or bit fields of fixed size. The record structures can be organized into trees or graphs. An instruction set can be designed for programs expressed as such trees. A processor can interpret these trees in a recursive fashion, and provide automatic storage management for the record structures.

We describe here the basic ideas behind the architecture, and for concreteness give a specific instruction set (on which variations are certainly possible). We also discuss the similarities and differences between these ideas and those of traditional architectures.

We discuss a prototype VLSI microprocessor which has been designed and fabricated. It is a small-scale version of the ideas presented here, containing a sufficiently complete instruction interpreter to execute small programs, and a rudimentary storage allocator. We are continuing to design and fabricate further implementations of these ideas.

Key Words and Phrases: microprocessors, large scale integration, integrated circuits, VLSI, list structure, linked lists, garbage collection, storage management, direct execution, high-level language architectures, interpreters, tail recursion, LISP, SCHEME

CR Categories: 4.13, 4.21 4.22, 4.34, 6.21, 6.22, 6.33

This work was supported in part by the National Science Foundation under Grant MCS77-04828, and in part by Air Force Office of Scientific Research Grant AFOSR-78-3593.

Introduction

An idea which has increasingly gained attention is that computer architectures should reflect specific language structures to be supported. This is an old idea; one can see features in the machines of the 1960's intended to support COBOL, FORTRAN, ALGOL, and PL/I. More recently research has been conducted into architectures to support string or array processing as in SNOBOL or APL.

An older and by now well-accepted idea is that of the stored-program computer. In such a computer the program and the data reside in the same memory; that is, the program is itself data which can be manipulated as any other data by the processor. It is this idea which allows the implementation of such powerful and incestuous software as program editors, compilers, interpreters, linking loaders, debugging systems, etc.

One of the great failings of most high-level languages is that they have abandoned this idea. It is extremely difficult, for example, for a PL/I (PASCAL, FORTRAN, COBOL ...) program to manipulate PL/I (PASCAL, FORTRAN, COBOL ...) programs.

On the other hand, many of these high-level languages have introduced other powerful ideas not present in standard machine languages. Among these are (1) recursively defined, nested data structures; and (2) the use of functional composition to allow programs to contain expressions as well as (or instead of) statements. The LISP language in fact has both of these features. It is unusual among high-level languages in that it also explicitly supports the stored-program idea: LISP programs are represented in a standardized way as recursively defined, nested LISP data structures. By contrast with some APL implementations, for example, which allow programs to be represented as arrays of characters, LISP also reflects the structure of program expressions in the structure of the data which represents the program. (An array of APL characters must be parsed to determine the logical structure of the APL expressions represented by the array. Similar remarks apply to SNOBOL statements represented as SNOBOL strings.)

It is for this reason that LISP is often referred to as a "high-level machine language". As with standard stored-program machine languages, programs and data are made of the same stuff. In a standard machine, however, the "stuff" is a homogeneous, linear (ordered) vector of fixed-size bit fields; a program is represented as an ordered sequence of bit fields (instructions) within the overall vector. In LISP, the "stuff" is a heterogeneous, unordered set of records linked to form lists, trees, and graphs; a program is represented as a tree (a "parse tree" or "expression tree") of linked records (a subset of the overall set of records). Standard machines usually exploit the linear nature of the "stuff" through such mechanisms as indexing by additive offset and linearly advancing program counters. A computer based on LISP can similarly exploit tree structures.

The counterpart of indexing is component selection; the counterpart of linear instruction execution is evaluation of expressions by recursive tree-walk.

Just as the "linear vector" stored-program-computer model leads to a variety of specific architectures, so with the "linked record" model. For concreteness we present here one specific architecture based on the linked record model which has actually been constructed.

List Structure and Programs

One of the central ideas of the LISP language is that storage management should be completely invisible to the programmer, so that he need not concern himself with the issues involved. LISP is an object-oriented language, rather than a value-oriented language. The LISP programmer does not think of variables as the objects of interest, bins in which values can be held. Instead, each data item is itself an object, which can be examined and modified, and which has an identity independent of the variable(s) used to name it.

In this section we discuss LISP data structures at the conceptual level; the precise form of LISP data objects is not of concern here. Later we will discuss specific representations within the machine. LISP data is collectively referred to as "S-expressions" ("S" for "symbolic"). For our purposes we will need only the special cases of S-expressions called atoms and lists. An atom is an "indivisible" data object, which we denote by writing a string of letters and digits; if only digits are used, then the atom is considered to be a number. Many special characters such as "-", "+", "@", and "*" are considered to be letters; we will see below that it is not necessary to specially reserve them for use as operator symbols. A list is a (possibly empty) sequence of LISP data objects, notated by (recursively) notating the objects in order, between a set of parentheses and separated by blank space. A list of the atoms "FOO", "43", and "BAR" would be written "(FOO 43 BAR)". Notice that the definition of a list is recursive. For example,

```
(DEFINE SECOND (LAMBDA (X) (CAR (CDR X))))
```

is a list of three things: the atomic symbol DEFINE, the atomic symbol SECOND, and another list of three things LAMBDA, (X), and (CAR (CDR X)).

A convenient way to use lists to represent algebraic expressions is to use "Cambridge Polish" notation, essentially a parenthesized version of prefix Polish notation. Numeric constants are encoded as numeric atoms; variables are encoded as non-numeric atoms (which henceforth we will call symbols); and procedure invocations (function calls) are encoded as lists, where the first element of the list represents the procedure and the rest represent the arguments. For example, the algebraic expression "a*b+c*d" can be represented as "(+ (* a b) (* c d))". Notice that LISP does not need the usual precedence

rules concerning whether multiplication or addition is performed first; the parentheses (or rather, the structure of the lists) explicitly define the order. Also, all procedure invocations have a uniform syntax, no matter how many arguments are involved. Infix, superscript, and subscript notations are not used; thus the expression " $J_p(x^2+1)$ " would be written "(J p (+ (↑ x 2) 1))".

To encode a conditional expression "if p then x else y" we write:

```
(IF p x y)
```

Expressions are made into procedures (functions) by the use of Church's lambda-notation [Church 1941]. For example,

```
(LAMBDA (X Y) (+ (* 3 Y) X))
```

evaluates to a function of two arguments x and y which computes $3Y+X$. The list of names of variables after the LAMBDA indicates how the names of variables in the expression are to be matched positionally to supplied arguments when the function is applied.

We can also encode recursive LISP programs as list data. For example, to compute N factorial (N!):

```
(DEFINE FACTORIAL
  (LAMBDA (N)
    (IF (= N 0) 1
        (* N (FACTORIAL (- N 1))))))
```

Suppose that we now want to write a LISP program which will take such a data structure and perform some useful operation on it, such as determining the value of an algebraic expression represented as a list structure. We need some procedures for categorizing, decomposing, and constructing LISP data.

The predicate ATOM, when applied to a LISP datum, produces true when given an atom and false otherwise. The empty list ("()") is considered to be an atom. The predicate NULL is true of only the empty list; its argument need not be a list, but may be any LISP datum. The predicate NUMBERP is true of numbers and false of symbols and lists. The predicate EQ, when applied to two symbols, is true if the two atomic symbols are identical. It is false when applied to two distinct symbols, or to a symbol and any other datum.

The decomposition operators for lists are traditionally called CAR and CDR for historical reasons. CAR extracts the first element of a list, while CDR produces a list containing all elements but the first. Because compositions of CAR and CDR are commonly used in LISP, an abbreviation is provided: all the C's and R's in the middle can be squeezed out. For example, "(CDR (CDR (CAR (CDR X))))" can be written as "(CDDADR X)".

The construction operator `CONS`, given any datum and a list, produces a new list whose `car` is the datum and whose `cdr` is the given list; that is, `CONS` adds a new element to the front of a list. The operator `LIST` can take any number of arguments (a special feature), and produces a list of its arguments.

Notice that `CONS` (and `LIST`) conceptually create new data structures. As far as the LISP programmer is concerned, new data objects are available in endless supply. They can be conveniently called forth to serve some immediate purpose and discarded when they are no longer of use. While creation is explicit, discarding is not; a data object simply disappears into limbo when the program throws away all references (direct or indirect) to that object.

The immense freedom this gives the programmer may be seen by an example taken from current experience. A sort of error message familiar to most programmers is "too many nested DO loops" or "more than 200 declared arrays" or "symbol table overflow". Such messages typically arise within compilers or assemblers which were written in languages requiring data tables to be pre-allocated to some fixed length. The author of a compiler, for example, might well guess, "No one will ever use more than, say, ten nested DO loops; I'll double that for good measure, and make the nested-DO-loop-table 20 long." Inevitably, someone eventually finds some reason to write 21 nested DO loops, and finds that the compiler overflows its fixed table and issues an error message (or, worse yet, doesn't issue an error message!). On the other hand, had the compiler writer made the table 100 long or 1000 long, most of the time most of the memory space devoted to that table would be wasted.

A compiler written in LISP would be much more likely to keep a linked list of records describing each DO loop. Such a list could be grown at any time by creating a new record on demand and adding it to the list. In this way as many or as few records as needed could be accommodated.

Now one could certainly write a compiler in any language and provide such dynamic storage management with enough programming. The point is that LISP provides automatic storage management from the outset and encourages its use (in much the same way that FORTRAN provides floating-point numbers and encourages their use, even though the particular processor on which a FORTRAN program runs may or may not have floating-point hardware).

Using `CAR`, `CDR`, and `CONS`, we can now write some interesting programs in LISP to deal with LISP data. For example, we can write a program `APPEND`, which given two lists produces their concatenation as a new list:


```
(DEFINE APPEND
  (LAMBDA (X Y)
    (IF (NULL X) Y
        (CONS (CAR X) (APPEND (CDR X) Y)))))
```

Because LISP programs are represented as LISP data structures, there is a difficulty with representing constants. For example, suppose we want to determine whether or not the value of the variable `x` is the symbol `"foo"`. We might try writing:

```
(EQ X FOO)
```

This doesn't work. The occurrence of `"foo"` does not refer to the symbol `foo` as a constant; it is treated as a variable, just as `"x"` is.

The essential problem is that we want to be able to write any LISP datum as a constant in a program, but some data objects must be used to represent other things, such as variables and procedure invocations. To solve this problem we invent a new notation: `(QUOTE d)` in a program represents the constant datum `d`. Thus we can write our test as `"(EQ X (QUOTE FOO))"`. Similarly,

```
(APPEND X (LIST Y Z))
```

constructs a list from the values of `y` and `z`, and appends the result to the value of `x`, while

```
(APPEND X (QUOTE (LIST Y Z)))
```

appends to the value of `x` the constant list `"(LIST Y Z)"`. Because the `QUOTE` construction is used so frequently in LISP, we use an abbreviated notation: `'foo` (`"foo"` with a preceding quote-mark) is equivalent to `"(QUOTE FOO)"`. This is only a notational convenience; the two notations denote the same list.

A LISP Interpreter

Here we exhibit an interpreter for a dialect of LISP, written in that dialect (this fact makes this interpreter meta-circular [Reynolds 1972] — it can interpret itself, but of course must interpret certain constructs by using those constructs). This dialect is based on SCHEME [Sussman 1975] [Steele 1978a], and differs semantically from most other LISP dialects primarily in that free variables are lexically scoped (as in ALGOL and PASCAL) rather than dynamically scoped (as in APL). Also, SCHEME was intentionally designed to be a small version of LISP, which makes it most suitable for implementation by hardware. Finally, SCHEME permits a correct implementation of tail-recursion; this means that function calls can be used to express true iteration, because calling a function is equivalent in all respects to a `GO TO` [Steele 1977a]

[Hewitt 1977] [Steele 1977b] [Steele 1978b].

```
(DEFINE EVAL
  (LAMBDA (EXP ENV)
    (IF (ATOM EXP)
      (IF (NUMBERP EXP) EXP (VALUE EXP ENV))
      (IF (EQ (CAR EXP) 'QUOTE)
        (CADR EXP)
        (IF (EQ (CAR EXP) 'LAMBDA)
          (LIST '&PROCEDURE (CADR EXP) (CADDR EXP) ENV)
          (IF (EQ (CAR EXP) 'IF)
            (IF (EVAL (CADR EXP) ENV)
              (EVAL (CADDR EXP) ENV)
              (EVAL (CADDR EXP) ENV))
            (APPLY (EVAL (CAR EXP) ENV)
                    (EVLIS (CDR EXP) ENV))))))))))

(DEFINE APPLY
  (LAMBDA (FUN ARGS)
    (IF (PRIMOP FUN) (PRIMOP-APPLY FUN ARGS)
      (IF (EQ (CAR FUN) '&PROCEDURE)
        (EVAL (CADDR FUN)
              (BIND (CADR FUN) ARGS (CADDR FUN))))
      (ERROR))))

(DEFINE EVLIS
  (LAMBDA (ARGLIST ENV)
    (IF (NULL ARGLIST) '()
      (CONS (EVAL (CAR ARGLIST) ENV)
            (EVLIS (CDR ARGLIST) ENV)))))
```

The evaluator is divided into two conceptual components: EVAL and APPLY. EVAL classifies expressions and directs their evaluation. Simple expressions (such as constants and variables) can be evaluated directly. For the complex case of procedure invocations (technically called "combinations"), EVAL looks up the procedure definition, recursively evaluates the arguments (using EVLIS), and then calls APPLY. APPLY classifies procedures and directs their application. Simple procedures (primitive operators) are applied directly. For the complex case of user-defined procedures, APPLY uses BIND (see below) to add to the lexical environment, a kind of symbol table, of the procedure, by associating the formal parameters from the procedure definition with the actual argument values provided by EVAL. The body of the procedure definition is then passed to EVAL, along with the environment just constructed, which is used to determine the values of variables occurring in the body.

In more detail, EVAL is a case analysis on the structure of the S-expression EXP. If it is an atom, there are two subcases. Numeric atoms

evaluate to themselves. Atomic symbols, however, encode variables; the value associated with that symbol is extracted from the environment ENV using the function VALUE (see below).

If the expression to be evaluated is not atomic, then it may be a QUOTE form, a LAMBDA form, an IF form, or a combination. For a QUOTE form, EVAL extracts the S-expression constant using CADR. LAMBDA forms evaluate to procedure objects (here represented as lists whose cars are the atom "&PROCEDURE") containing the lexical environment and the "text" of the procedure definition. For an IF form, the predicate part is recursively evaluated; depending on whether the result is true or false, the consequent or alternative is selected for evaluation. For combinations, the procedure is obtained, the arguments evaluated (using EVLIS), and APPLY called as described earlier.

EVLIS is a simple recursive function which calls EVAL on successive arguments in ARGLIST and produces a list of the values in order.

APPLY distinguishes two kinds of procedures: primitive and user-defined. For now we avoid describing the precise implementation of primitive procedures by assuming the existence of a predicate PRIMOP which is true only of primitive procedures, and a function PRIMOP-APPLY which deals with the application of such primitive procedures. We consider primitive procedures to be a kind of atomic S-expression other than numbers and atomic symbols; we define no particular written notation for them here. However, primitive procedures are not to be confused with the atomic symbols used as their names. The actual procedure involved in the combination (CAR X) is not the atomic symbol CAR, but rather some bizarre object (the value of the atomic symbol CAR) which is meaningful only to PRIMOP-APPLY.

The interpreter uses several utility procedures for maintaining environments. An environment is represented as a list of buckets; each bucket is a list whose car is a list of names and whose cdr is a list of corresponding values. (Note that this representation is not the same as the association list ("a-list") representation traditionally used in LISP interpreters.) If a variable name occurs in more than one bucket, the most recently added such bucket has priority; in this way new symbol definitions added to the front of the list can supersede old ones. In this way the "shadowing" effect of lexical block structure is obtained. The code for manipulating environments is below.

```
(DEFINE BIND
  (LAMBDA (VARS ARGS ENV)
    (IF (= (LENGTH VARS) (LENGTH ARGS))
        (CONS (CONS VARS ARGS) ENV)
        (ERROR))))
```

```
(DEFINE VALUE
  (LAMBDA (NAME ENV)
    (VALUE1 NAME (LOOKUP NAME ENV))))

(DEFINE VALUE1
  (LAMBDA (NAME SLOT)
    (IF (EQ SLOT '&UNBOUND) (ERROR)
        (CAR SLOT))))

(DEFINE LOOKUP
  (LAMBDA (NAME ENV)
    (IF (NULL ENV) '&UNBOUND
        (LOOKUP1 NAME (CAAR ENV) (CDAR ENV) ENV))))

(DEFINE LOOKUP1
  (LAMBDA (NAME VARS VALS ENV)
    (IF (NULL VARS) (LOOKUP NAME (CDR ENV))
        (IF (EQ NAME (CAR VARS)) VALS
            (LOOKUP1 NAME (CDR VARS) (CDR VALS) ENV)))))
```

BIND takes a list of names, a list of values, and a symbol table, and produces a new symbol table which is the old one augmented by an extra bucket containing the new set of associations. (It also performs a useful error check — **LENGTH** returns the length of a list.)

VALUE is essentially an interface to **LOOKUP**. The check for **&UNBOUND** catches incorrect references to undefined variables.

LOOKUP takes a name and a symbol table, and returns that portion of a bucket whose car is the associated value.

State-Machine Implementation

The LISP interpreter we have presented is recursive. It implicitly relies on a hidden control mechanism which retains the state information which must be saved for each recursive invocation. In this section we make this control information explicit. Below we present an interpreter in the form of a state machine controller. The controller manipulates a small set of registers, and also issues commands to a list memory system. The recursion control information which is typically kept on a stack will be maintained in the linked-list memory.

This evaluator, written in LISP, has five global variables which are used to simulate the registers of a machine. **EXP** is used to hold the expression or parts of the expression under evaluation. **ENV** is used to hold the pointer to the environment structure which is the context of evaluation of the current expression. **VAL** is used to hold the value developed in evaluation

of expressions. It is set whenever a primitive operator is invoked, or whenever a variable is evaluated, a quoted expression is evaluated, or a lambda expression is evaluated. ARGS is used to hold the list of evaluated arguments (the "actual parameters") being accumulated for a combination. Finally, CLINK is the pointer to the top of the list structure which is the control stack. (It is called "CLINK" for historical reasons stemming from CONNIVER [McDermott 1974] and "spaghetti stacks" [Bobrow 1973].)

The style of coding here depends on "tail-recursion"; that is, iterative loops are implemented as patterns of function calls. When the last thing a function does is to call another function, then the effect is simply to perform a GO TO to the called function, passing arguments if any, without pushing anything onto a stack, not even a return address (for in this situation it is not necessary to return). [Hewitt 1977] [Steele 1977b]

EVAL-DISPATCH is the procedure which dispatches on the type of an expression — implementing the action of EVAL. When EVAL-DISPATCH is called, EXP contains an expression to be evaluated, ENV contains the environment for the evaluation, and the top element of CLINK is a "return address", i.e., the name of a function to call when the value has been determined and placed in VAL.

```
(DEFUN EVAL-DISPATCH ()
  (COND ((ATOM EXP)                                ;If an atomic expression:
        (COND ((NUMBERP EXP)                       ; numbers evaluate
              (SETQ VAL EXP)                        ; to themselves
              (POPJ-RETURN))                        ; (i.e., are "self-quoting"),
              (T                                     ; but symbols must be looked
              (SETQ VAL (VALUE EXP ENV))            ; up in the environment.
              (POPJ-RETURN))))
        ((EQ (CAR EXP) 'QUOTE)                    ;If a QUOTE expression
         (SETQ VAL (CADR EXP))                    ; extract the quoted constant
         (POPJ-RETURN))                          ; and return it.
        ((EQ (CAR EXP) 'LAMBDA)                   ;If a LAMBDA expression
         (SETQ VAL (CADR EXP))                    ; get the formal parameters,
         (SETQ EXP (CADDR EXP))                  ; get the body,
         (SETQ VAL (LIST '&PROCEDURE VAL EXP ENV)) ; and construct a closure
         (POPJ-RETURN))                          ; which includes ENV.
        ((EQ (CAR EXP) 'IF)                       ;If a conditional,
         (SETQ CLINK (CONS ENV CLINK))            ; save the environment
         (SETQ CLINK (CONS EXP CLINK))            ; save the expression,
         (SETQ CLINK (CONS 'EVIF-DECIDE CLINK))    ; set up a return address,
         (SETQ EXP (CADR EXP))                    ; then extract the predicate
         (EVAL-DISPATCH))                        ; and evaluate it.
```



```

((NULL (CDR EXP))                ;If a call with no arguments,
 (SETQ CLINK (CONS 'APPLY-NO-ARGS CLINK)) ; set up a return address,
 (SETQ EXP (CAR EXP))             ; get the function position
 (EVAL-DISPATCH))               ; and evaluate it.
(T                               ;Otherwise,
 (SETQ CLINK (CONS ENV CLINK))    ; save ENV,
 (SETQ CLINK (CONS EXP CLINK))    ; save EXP,
 (SETQ CLINK (CONS 'EVARGS CLINK)) ; set up return address,
 (SETQ EXP (CAR EXP))            ; get the function position
 (EVAL-DISPATCH))))            ; and evaluate it.

```

When the process evolved by the evaluator has finished the evaluation of a subexpression, it must continue executing the rest of the expression. The place in the evaluator to continue executing was pushed onto CLINK when the evaluation of the subexpression was begun. This return address is now at the top of the CLINK, where it can be popped off and called:

```

(DEFUN POPJ-RETURN ()            ;Return to caller:
 (SETQ EXP (CAR CLINK))          ; Save return address in EXP,
 (SETQ CLINK (CDR CLINK))        ; and pop it off CLINK.
 (FUNCALL EXP))                 ; Transfer control.

```

After the predicate part of a conditional is evaluated, the process comes back to here to look at VAL to see whether the consequent or the alternative branch is to be taken. One of these is selected and made the EXP to be further evaluated.

```

(DEFUN EVIF-DECIDE ()
 (SETQ EXP (CAR CLINK))          ;Restore expression
 (SETQ CLINK (CDR CLINK))        ; and pop it off.
 (SETQ ENV (CAR CLINK))          ;Restore ENV
 (SETQ CLINK (CDR CLINK))        ; and pop it off.
 (COND (VAL                      ;If predicate was true,
        (SETQ EXP (CADDR EXP))) ; extract consequent.
        (T                      ;Otherwise
         (SETQ EXP (CADDDR EXP)))) ; extract alternative.
 (EVAL-DISPATCH))              ;In either case, evaluate it.

```

The following procedures are the states the evaluator must go through to evaluate the arguments to procedures before applying them. There is a special-case check in EVAL-DISPATCH for functions with no arguments. In this case, it is not necessary to save the state of the evaluator when evaluating the function position because there are no further arguments to evaluate. One may just apply the procedure which comes back in VAL. This is a case of "evlis tail-recursion" (see [Wand 1977]). We will see this idea again in EVARGS1 where we have a special-case check for evaluation of the last argument.

```

(DEFUN APPLY-NO-ARGS ()
  (SETQ ARGS NIL)           ;Set up null argument list
  (SAPPLY))                 ; and apply function in VAL.

```

General argument evaluations come to EVARGS. This segment of the evaluator incorporates some cleverness in that it checks for the special case of the last argument in a combination. However, for the sake of clarity and uniformity we did not try to remove all unnecessary pushing and popping. There are many cleverer ways to write this code, as we will see later. The following procedure is the initialization of the argument evaluation loop.

```

(DEFUN EVARGS ()
  (SETQ EXP (CAR CLINK))      ;Restore EXP
  (SETQ CLINK (CDR CLINK))    ; and pop it off.
  (SETQ ENV (CAR CLINK))      ;Restore ENV,
  (SETQ CLINK (CDR CLINK))    ; and pop it.
  (SETQ CLINK (CONS VAL CLINK)) ;Save function.
  (SETQ EXP (CDR EXP))        ;Get rid of function part.
  (SETQ ARGS NIL)            ;Initialize argument list.
  (EVARGS1))                  ;Evaluate arguments.

```

The function EVARGS1 is the top of the argument evaluation loop.

```

(DEFUN EVARGS1 ()
  (COND ((NULL (CDR EXP))      ;Is this the last argument?
    (SETQ CLINK (CONS ARGS CLINK)) ;If so, save argument list,
    (SETQ CLINK (CONS 'LAST-ARG CLINK)) ; set up return address,
    (SETQ EXP (CAR EXP))        ; set up last argument,
    (EVAL-DISPATCH))          ; and evaluate it.
    (T                          ;Otherwise,
      (SETQ CLINK (CONS ENV CLINK)) ; save ENV,
      (SETQ CLINK (CONS EXP CLINK)) ; save EXP,
      (SETQ CLINK (CONS ARGS CLINK)) ; save argument list,
      (SETQ CLINK (CONS 'EVARGS2 CLINK)) ; set up return address,
      (SETQ EXP (CAR EXP))        ; set up next argument,
      (EVAL-DISPATCH))))        ; and evaluate it.

```

The function EVARGS2 is where we end up after each argument is evaluated. The evaluated argument is accumulated into (consed onto) the list growing in ARGS.

```

(DEFUN EVARGS2 ()
  (SETQ ARGS (CAR CLINK))           ;Restore argument list,
  (SETQ CLINK (CDR CLINK))          ; and pop it off.
  (SETQ EXP (CAR CLINK))            ;Restore EXP,
  (SETQ CLINK (CDR CLINK))          ; and pop it off.
  (SETQ ENV (CAR CLINK))            ;Restore ENV,
  (SETQ CLINK (CDR CLINK))          ; and pop it off.
  (SETQ ARGS (CONS VAL ARGS))       ;Add value to argument list.
  (SETQ EXP (CDR EXP))              ;Flush form just evaluated.
  (EVARGS1))                        ;Go evaluate next argument.

```

When the last argument has been evaluated we come back here. The value is accumulated onto the ARGS and the function is restored from the stack. Execution continues at SAPPLY for application of the procedure to the evaluated arguments.

```

(DEFUN LAST-ARG ()
  (SETQ ARGS (CAR CLINK))           ;Restore argument list,
  (SETQ CLINK (CDR CLINK))          ; and pop it off.
  (SETQ ARGS (CONS VAL ARGS))       ;Add last value to it.
  (SETQ VAL (CAR CLINK))            ;Retrieve function,
  (SETQ CLINK (CDR CLINK))          ; and pop it off.
  (SAPPLY))                        ;Apply function to arguments.

```

SAPPLY is the state machine analog of APPLY. This procedure checks out what kind of procedure is to be applied. If it is primitive, the appropriate actions occur. If it is a procedural closure, we evaluate the body of the closed procedure in an environment constructed by binding the formal parameters of the closed procedure to the actual parameters (in ARGS) in the environment carried in the closure.

```

(DEFUN SAPPLY ()
  (COND ((PRIMOP? VAL)              ;Apply function in VAL to ARGS.
    (SETQ VAL (PRIMOP-APPLY VAL ARGS)) ;If a primitive procedure,
    (POPJ-RETURN))                  ; do it!
    ((EQ (CAR VAL) '&PROCEDURE)      ; then return value to caller.
    (SETQ ENV
      (BIND (CADR VAL)
        ARGS
        (CADDR VAL)))              ;If a defined procedure,
    (SETQ EXP (CADDR VAL))          ; set up its environment
    (EVAL-DISPATCH))              ; by binding the formals
    (T (ERROR))))                  ; to the actuals
                                   ; in the closure environment
                                   ; then get the procedure body
                                   ; and evaluate it.
                                   ;Otherwise, error.

```

In this state-machine code we have avoided functional composition. Each statement is an assignment or a conditional. (We have used the usual LISP COND conditional form, rather than IF, for reasons of convenience. This interpreter is not meta-circular. Instead, it is working MacLISP [Moon 1974]

code which implements a non-MacLISP version of LISP.) An assignment can contain at most one call to a storage management procedure such as CONS or CAR (we allow calls to, e.g., CADDR, realizing that (SETQ X (CADDR Y)) can be considered an abbreviation for the sequence (SETQ X (CDR Y)), (SETQ X (CDR X)), (SETQ X (CAR X))). Also, VALUE and BIND can be considered here to be complex storage operations (defined essentially as before).

Representing List Data

Lists are normally represented by records each of which contains two pointers to other records. One pointer is the car, and the other is the cdr. In this way a list (A (B C) D) can be visualized by the diagram in Figure 1.

Atoms are represented as records of other types.

The exact representation of a pointer is not of interest here. All we really care about is that if we give the pointer to the memory system, it can return the contents of the record pointed to. (In particular, there is nothing at this level requiring the parts of a record to be "contiguous". Later we will discuss ways to represent LISP data within standard linear memories.)

In our particular architecture, we find it convenient to associate with each pointer a type field describing the nature of the record pointed to. (This use of typed pointers was inspired by the implementations of such languages as ECL [Wegbreit 1974] and MUDDLE (now called MDL) [Galley 1975].) This type field can be exploited for other purposes as well; in particular, we shall use it to encode "opcodes" and "return addresses". We will say that the type field is a part of the pointer, and that the other part of the pointer (that which identifies another record) is the address part. The list shown above, with type fields added is shown in Figure 2.

One efficiency problem with the version of the LISP interpreter given above is that the repeated consecutive tests for atoms, LAMBDA, IF, etc. take time. Conceptually what these tests are doing is a dispatch on the syntactic category of the expression. Each expression is distinguished by a special symbol in the car position — except for atoms and procedure calls. The evaluator could be made faster and simpler if it could dispatch in a more uniform way.

Another efficiency problem is that LOOKUP must search for the values of variables. Because our dialect of LISP is lexically scoped like ALGOL, we can arrange for variable references to specify "n levels back, and then j over" in much the same way used by the ALGOL "display" technique, eliminating the search.

To allow these efficiencies we systematically alter the representation of programs. They will still be represented as trees of list records, but we encode the syntactic categories in the type fields of pointers to the expressions. EVAL can then simply dispatch on this type code. For a pointer whose type is "variable reference", we use the address part as "immediate data" indicating the values of n and j for the environment lookup. The representation of the following program fragment

```
(IF A '(X Y) (IF C 'D (CONS E 69)))
```

is illustrated in Figure 3.

Because variable references and other constructs have separate types, lists and symbols can be "self-evaluating" in the same way as numbers. Also, we assume that CONS is a "built-in" primitive operator, and encode that operator in the type field at the end of the list representing the call to CONS. The encoding of IF forms has been changed to reduce space requirements; the car of a conditional is the predicate, and the cdr is a cons cell whose car is the consequent and whose cdr is the alternative.

We ought perhaps to define a printed representation for all these new data types. We do not do this here, however. We assume that in practice one will write LISP code in the usual style, and a simple "compiler" program will transform it into the typed-pointer representation. (The typed-pointer representation is isomorphic to the original expression. It is easy for another program to reverse-compile a typed-pointer expression to reconstruct the original code. There is a problem with recovering the names of variables, but this can be taken care of by associating with each procedure a description of the names of its variables.)

To describe the evaluator for this new representation, we introduce a construct TYPE-DISPATCH (essentially a CASE statement on the type) which is not part of the LISP language, but which we use to indicate the underlying mechanism. The names in double quotes can be thought of as values of some PASCAL-style enumeration type; they represent the distinct values which can occur in the type field. We also use a primitive operator TYPED-CONS, which creates a new list-like cell with a specified type. The new evaluator is very much like the old one: EVCOMB is sort of like EVLIS combined with the entry point to APPLY. The function DISPLAY-BIND adds a new contour to an ALGOL-style display structure; DISPLAY-LOOKUP accesses a value in such a display.

```

(DEFINE EVAL
  (LAMBDA (EXP ENV)
    (TYPE-DISPATCH EXP
      ("NUMBER" EXP)
      ("SYMBOL" EXP)
      ("LIST" EXP)
      ("VARIABLE" (DISPLAY-LOOKUP EXP ENV))
      ("PROCEDURE" (TYPED-CONS "CLOSURE" (CDR EXP) ENV))
      ("IF" (IF (EVAL (CAR EXP) ENV)
                (EVAL (CADR EXP) ENV)
                (EVAL (CDDR EXP) ENV)))
      ("COMBINATION" (EVCOMB (CDR EXP)
                             ENV
                             (TYPED-CONS "LIST"
                                           (EVAL (CAR EXP) ENV)
                                           '()))))
      (OTHERWISE (ERROR))))))

(DEFINE EVCOMB
  (LAMBDA (EXP ENV ARGS)
    (TYPE-DISPATCH EXP
      ("MORE-ARGS" (EVCOMB (CDR EXP)
                          ENV
                          (TYPED-CONS "LIST"
                                      (EVAL (CAR EXP) ENV)
                                      ARGS)))
      (OTHERWISE (APPLY EXP ARGS))))))

(DEFINE APPLY
  (LAMBDA (FUN ARGS)
    (TYPE-DISPATCH FUN
      ("FUNCALL" (EVAL (CDAAR ARGS)
                      (DISPLAY-BIND (CDR ARGS)
                                    (CDAR ARGS))))
      ("CONS" (CONS (CADR ARGS) (CAR ARGS)))
      ("CAR" (CAAR ARGS))
      ("CDR" (CDAR ARGS))
      ("ATOM" (ATOM (CAR ARGS)))
      ...
      (OTHERWISE (ERROR))))))

```

When a non-"MORE-ARGS" type code is seen in EVCOMB, it indicates that a primitive operation is to be performed on the argument values. EVCOMB then calls APPLY to perform this operation. (As shown here, APPLY needlessly duplicates the dispatching operation in EVCOMB; we have done this to exhibit the similarity of this interpreter to the previous one. Later we will remove this duplication.) One of these primitive operations, "FUNCALL", is used to invoke user-defined procedures (closures).

The type codes used to indicate primitive operations such as "CONS" may overlap those used to distinguish syntactic categories, because they are used in different contexts. Compare this to the way in which the same bits in an instruction can be used for different purposes depending on the opcode; for example, in the PDP-11 the same bits of an instruction word can be a register number, part of a branch offset, or condition code bits.

Combining these Ideas

The state machine implementation of a LISP interpreter can be combined with the typed pointer dispatch idea to form a very efficient interpreter for LISP which can be easily implemented directly in hardware. We now present such an interpreter, written in a statement-oriented language to emphasize that we are describing a hardware interpreter. As before, the controller manipulates a small set of registers, and also issues commands to a list memory system. The recursion-control information is, as before, stored in a push-down control list maintained in linked-list memory. Type fields in the cdr pointers of the control list will be used to retain "return addresses" within the state machine; in this way return addresses do not require any extra conses in the CLINK. (Compare this with the previous state-machine interpreter, which used separate tokens in the CLINK as return addresses.) This is possible because the set of return addresses is small.

```
BEGIN "EVALUATOR"
  DECLARE REGISTERS
    EXP          !GENERALLY HOLDS EXPRESSION BEING EVALUATED
    ENV          !HOLDS CURRENT ENVIRONMENT
    VAL          !RESULT OF EVALUATION; ALSO SCRATCH
    ARGS        !ACCUMULATES EVALUATED ARGUMENTS OF A COMBINATION
    CLINK        !"CONTROL LINK": RECURSION CONTROL STACK
  EVAL:  TYPE-DISPATCH ON EXP INTO
    "NUMBER": GOTO SELF
    "SYMBOL": GOTO SELF
    "LIST": GOTO SELF
    "VARIABLE": GOTO LOOKUP
    "PROCEDURE": GOTO PROC
    "IF": GOTO IF1
    "COMBINATION": GOTO EVCOMB
    HCTAPSID-EPYT
  SELF:  VAL := EXP; GOTO RETURN
  PROC:  VAL := TYPED-CONS("CLOSURE", EXP, ENV); GOTO RETURN
  IF1:   VAL := CDR(EXP)
        CLINK := CONS(ENV, CLINK)
        CLINK := TYPED-CONS("IF2", VAL, CLINK)
        EXP := CAR(EXP); GOTO EVAL
```



```

!RECURSIVE EVALUATION OF PREDICATE RETURNS HERE
IF2:   EXP := CAR(CLINK)
       CLINK := CDR(CLINK)
       ENV := CAR(CLINK)
       CLINK := CDR(CLINK)
       IF NULL(VAL)
           THEN EXP := CDR(EXP); GOTO EVAL
           ELSE EXP := CAR(EXP); GOTO EVAL
       FI
EVCOMB: ARGS := '()
EVCOM1: TYPE-DISPATCH ON EXP INTO
        "COMBINATION": GOTO EVCOM2
        "FUNCALL": GOTO CALL
        "CONS": GOTO CONS
        "CAR": GOTO CAR
        "CDR": GOTO CDR
        ...
        HCTAPSID-EPYT
EVCOM2: CLINK := CONS(ENV, CLINK)
       CLINK := CONS(ARGS, CLINK)
       VAL := CDR(EXP)
       CLINK := TYPED-CONS("EVCOM3", VAL, CLINK)
       EXP := CAR(EXP); GOTO EVAL
!RECURSIVE EVALUATION OF ARGUMENT RETURNS HERE
EVCOM3: EXP := CAR(CLINK)           !UNWIND STACK
       CLINK := CDR(CLINK)
       ARGS := CAR(CLINK)
       CLINK := CDR(CLINK)
       ENV := CAR(CLINK)
       CLINK := CDR(CLINK)
       ARGS := CONS(VAL, ARGS); GOTO EVCOM1
CALL:   EXP := CAR(VAL)             !N.B. VAL = CAR(ARGS)
       VAL := CDR(VAL)
       ENV := CONS(ARGS, VAL); GOTO EVAL
CONS:   ARGS := CDR(ARGS)           !I.E., ARGS := CADR(ARGS)
       ARGS := CAR(ARGS)           !(ALREADY HAD VAL := CAR(ARGS), IN EFFECT)
       VAL := CONS(ARGS, VAL); GOTO RETURN
CAR:    VAL := CAR(VAL); GOTO RETURN
CDR:    VAL := CDR(VAL); GOTO RETURN
...
RETURN: TYPE-DISPATCH ON CLINK INTO
        "IF2": GOTO IF2
        "EVCOM3": GOTO EVCOM3
        HCTAPSID-EPYT           !SESOL ARTSKJID
END "EVALUATOR"

```

In this state-machine code we have avoided functional composition rigorously. Each statement is an assignment or a dispatch operation

(IF-THEN-ELSE being a kind of dispatch). As assignment can contain at most one call to a simple storage management procedure such as CONS or CAR. Each statement goes to another statement (to the one textually following, if no GOTO clause is present).

We have omitted the details of the LOOKUP operation (it gets the value from the environment and then goes to RETURN). We have, however, shown DISPLAY-BIND (beginning at CALL). These are not done as subroutines (as they were in the previous state-machine interpreter); they are coded "in-line" as state-machine code.

Recursive evaluation of subexpressions is handled by using an explicit stack. When for an IF or a COMBINATION a recursive evaluation is needed, any required registers (e.g., ENV) are consed onto the control structure CLINK. The last cons onto CLINK uses the type code to encode the "return address" (IF2 or EVCOM3) within the state machine. (These return address codes may be the same codes used as "opcodes" or "primitive operator codes" — this is a third, distinct context in which type bits are used for some funny purpose unrelated to the type of the data.) The expression to be recursively evaluated is put into EXP, and then state EVAL is entered. When the evaluation finishes, the code at RETURN decodes the type field of CLINK and resumes execution of the caller, which retrieves the saved information from CLINK and carries on. Thus CLINK, though implemented as linked records, behaves as a stack.

Storage Management

A complete LISP system, as implied in the previous section, is conveniently divided into two parts: (1) a storage system, which provides an operator for the creation of new data objects and also other operators (such as pointer traversal) on those objects; and (2) a program interpreter (EVAL), which executes programs expressed as data structures within the storage system. (Note that this memory/processor division characterizes the usual von Neumann architecture also. The differences occur in the nature of the processor and the memory system.)

Most hardware memory systems which are currently available commercially are not organized as sets of linked lists, but rather as the usual linearly-indexed vectors. (More precisely, commercially available RAMs are organized as Boolean N-cubes indexed by bit vectors. The usual practice is to impose a total ordering on the memory cells by ordering their bit-vector addresses lexicographically, and then to exploit this total ordering by using indexing hardware typically containing an addition unit (or, more rarely, a subtraction unit, as on the IBM 7094).)

Commercially available memories are, moreover, available only in finite sizes (more's the pity). Now the free and wasteful throw-away use of data objects would cause no problem if infinite memory were available, but

within a finite memory it is an ecological disaster. In order to make such memories useable to our processor we must interpose between EVAL and the storage system a storage manager which makes a finite vector memory appear to the evaluation mechanism to be an infinite linked-record memory. This would seem impossible, and it is. The catch is that at no time may more records be active than will fit into the finite memory actually provided. The memory is "apparently infinite" in the sense that an indefinitely large number of new records can be "created" using the CONS operator. The storage manager recycles discarded records in order to create new ones in a manner completely invisible to the evaluator.

The storage manager therefore consists of routines which implement the operations CAR, CDR, CONS, etc. in terms of the vector memory, plus a garbage collector which deals with the finiteness of the memory by locating records which have been discarded and making them available to the CONS routine for recycling.

The method we use for implementing CAR, CDR, and CONS is the usual one of using two consecutive words of linear memory to hold a list cell, one being the car and the other the cdr, where each word of memory can hold a type field and an address field. The address part of a pointer is in turn the address within the linear memory of the record pointed to. (This may seem obvious, but remember that until now we have been noncommittal about the precise representation of pointers, as until this point all that was necessary was that the memory system associate records with pointers by any convenient means whatsoever. The evaluator is completely unconcerned with the format or meaning of addresses; it merely accepts them from the memory system and eventually gives them back later to retrieve record components. One may think of an address as a capability for accessing a record using certain defined operations.)

Many techniques for garbage collection are well-documented in the literature [McCarthy 1962] [Minsky 1963] [Hart 1964] [Saunders 1964] [Schorr 1967] [Knuth 1968] [Conrad 1974] [Deutsch 1976] [Gries 1977] [Baker 1978] [Morris 1978], and will not be discussed here. Suffice it to say here that the storage manager can be implemented as a second state machine. The storage manager runs continuously, performing services for the evaluator. When the storage manager has completed a request, it then advances the evaluator to its next state, and dispatches on the new request from the evaluator. The storage manager can also read from or write into the off-chip memory.

Our Prototype Microprocessor

In 1978 we used the ideas of the preceding sections to implement a prototype LISP evaluator in the form of a VLSI microprocessor. This prototype was originally called SIMPLE (Small Integrated Micro-Processor for Lisp

Expressions), and later renamed SCHEME-78. This implementation was intended only to test the concepts involved, and is not a complete working microprocessor. It has 11-bit words, divided into 3 type bits and 8 address bits.

Internally, the processor contains two state machines, EVAL and GC. Each machine has several registers (five for EVAL, four for GC) on a common bus (E bus, G bus). Each state machine is in the form of a read-only memory plus a "micro-PC" which encodes the current state. At each transition a state machine can read one register onto its bus, load one or more other registers from the bus, request some storage operation to occur, and enter some new state (possibly computed by dispatching on bits obtained from the bus). The storage operations requested by the EVAL machine are relayed to the GC machine, and requests from the GC machine are sent to the external memory system. The E bus and G bus can be connected together; this is done to pass pointers between EVAL and GC. Similarly, the G bus and pads can be connected; this is done to pass addresses and data between GC and the external memory.

When the chip is initialized, it takes a given expression and evaluates it in the global (null) environment. If that evaluation ever terminates, the result of the evaluation is stored in memory and the chip halts, with the evaluator looping in a dead state. The chip also halts if it runs out of memory (i.e., after consuming the last of the 256 words), with the storage manager looping in a dead state (see below).

The type field can hold one of eight "op codes":

0 = constant list	4 = procedure
1 = constant symbol	5 = conditional (if-then-else)
2 = variable reference	6 = procedure call
3 = constant closure	7 = quoted constant

The evaluation of a type 0 (list), 1 (symbol), or 3 (closure) object simply results in that object; such objects are "self-evaluating". (Notice that symbols are not the same as variables here; this usage has been "compiled out".)

The evaluation of type 7 (quote) returns the cdr of the object. In this way any object whatsoever, not just a list, symbol, or closure, can be included as a constant datum in a program. (Given this operation, opcodes 0, 1, and 3 are functionally redundant; however, they are faster and occur frequently in programs. Also, these codes distinguish between types of data when used as data types per se.)

The evaluation of type 2 (variable reference) treats the address part of the pointer as a two's-complement number, which is the negative of the number of times to cdr the current environment; the car of this is then the value of the variable. (To simplify the lookup hardware (at the expense of

speed) environments are maintained as a simple flat list of values; the contours are concatenated rather than chained.)

The evaluation of type 4 (procedure) results in a pointer to a newly-allocated word pair. This pointer has type 3 (closure). The car of the pair contains the cdr of the procedure; this is the code body of the procedure. The cdr of the pair contains the current environment (the environment within which the procedure object is being evaluated). In this way the code and the environment are bound up together (as a closure, or "FUNARG" [Moses 1970]) for later application. The car of the procedure object is not used by the evaluator. The compiler can use this slot to store documentation about the procedure, such as the name of the procedure, the name of the file it was read from (for the text editor), the names of its variables (for the decompiler and run-time debugger), etc.

A conditional (type 5) points to a two-word cell, the cdr of which points to another two-word cell. The car of the conditional object is a predicate expression (IF), the cadr is a consequent expression (THEN), and the caddr is an alternative expression (ELSE). The predicate expression is evaluated first; depending on whether the result is non-NIL or NIL, then the consequent or alternative is evaluated, throwing away the other one, to produce the value of the conditional.

A procedure call (type 6) is the most complicated of the lot. It is a list of indefinite length, chained together by cdr pointers. Each cdr pointer except the last must have type 0 (list). The last cdr pointer should have a zero address and a non-zero type. This last type specifies the operation to be performed. In cdring down the list, SIMPLE evaluates each of the expressions in the car, saving the resulting values. These values are available as arguments to the operation to be performed. The operations available are:

0 = <more arguments>	4 = ATOM
1 = CAR	5 = PROGN
2 = CDR	6 = LIST
3 = CONS	7 = FUNCALL

For operations CAR, CDR, and ATOM there should be one argument; for CONS, two. No checking is performed for this. For PROGN, LIST, and FUNCALL there may be any non-zero number of arguments.

CAR, CDR, CONS, ATOM, and LIST are the standard LISP primitive operations. LIST is actually a REVERSE-LIST, because it produces a list of the arguments in reverse order (the implementation is trivial, taking advantage of the fact that the interpreter must cons up a list of the arguments anyway — but this implementation happens to produce the list in a reversed order). This matters only if the calculations of the arguments have side effects which could interfere with each other. PROGN is a standard LISP

primitive which evaluates any number of arguments and returns only the last one. This is useful only when side effects are used. Even though no primitive side effects are provided in the prototype instruction set, PROG_N was included to test that idea.

FUNCALL is the operation which calls a user procedure (actually a closure). The last argument (not the first) must be a closure, which is applied to the preceding arguments. The body (car) of the closure is evaluated in an environment produced by tacking (NCONCing, in LISP parlance) all the arguments onto the front of the environment (cdr) of the closure. Because successive sets of variables are tacked together using NCONC rather than being consed onto a display, the environment in the prototype processor takes the form of a simple list of values rather than a list of buckets of values. As mentioned above, this is done so that a variable reference can be simply "n back" rather than "n back and j over". Notice that the closure itself is added to the environment along with the other arguments. In this way the procedure can refer to itself recursively. This idea was borrowed from [Backus 1978].

As an example, an expression calling for the evaluation of the expression

```
(APPEND '(A B C) '(D E F))
```

including the definition of APPEND itself, is shown in Figure 4.

The GC processor executes requests from the EVAL processor. Each request specifies an operation, and can also transmit or receive one pointer on the E-bus/G-bus connection. Operations such as CAR, CDR, CONS, and RPLACD are provided. Because only one operand can be passed at a time to the storage manager an operation such as CAR is actually managed as two operations:

- (1) pass operand to storage manager and request CAR;
- (2) retrieve result of storage operation.

Similarly, CONS is managed as three operations:

- (1) pass the cdr part to storage manager;
- (2) pass the car part, and request CONS;
- (3) retrieve result.

Often sequences of operations can be optimized; for example, after requesting a CAR, the result need not be retrieved if it is to be used immediately as one operand of a CONS. In this case (CONS (CAR X) C) takes only three transactions, not five. The storage manager provided variants of operations such as CAR which specify whether or not to pass the result back, or (for CONS) whether an operand will be passed or is to be taken from some previous result.

In point of fact, in the prototype processor, the storage manager includes no garbage collector. The prototype was one project of a "project set" including some two dozen separate circuits, all of which had to be fit onto a single chip together. This imposed severe area limitations which

restricted the address size to eight bits, and required the elimination of the microcode for the garbage collector. However, space for microcode was the only limitation; all the other necessary structures for garbage collection were in the design. The complexity of a full garbage collector is comparable to that of the evaluator shown above.

Physical Layout of the Prototype Processor

The evaluator and the storage manager are each implemented in the same way as an individual processor. Each processor has a state-machine controller and a set of registers. On each clock cycle the state-machine outputs control signals for the registers and also makes a transition to a new state.

The contents of any register is a pointer, containing an address field (8 bits in the prototype) and a type field (3 bits in the prototype). The registers of a processor are connected by a common bus (E bus in the evaluator, G bus in the storage manager). Signals from the controller can read at most one register onto the bus, and load one or more other registers from the bus. One register in each controller has associated incrementation logic; the controller can cause the contents of that register, with 1 added to its address part, to be read onto the bus. The controller can also force certain constant values onto the bus rather than reading a register.

The processors can communicate with each other by causing the E and G busses to be connected. The address and type parts of the busses can be connected separately. (Typically the E bus might have its address part driven from the G bus and its type part driven by a constant supplied by the evaluator controller. This is used to do the TYPED-CONS operation.) The G bus can also be connected to the address/data lines for the off-chip memory system. The storage-manager controller produces additional signals (ADR and WRITE) to control the external memory. In a similar manner, the evaluator controller produces signals which control the storage manager. (Remember that from the point of view of the evaluator, the storage manager is the memory interface!)

Each controller effectively has an extra "state register" which may be thought of as its "micro-PC". At each step the next state is computed by combining its current state with external signals in the following manner. Each "microinstruction" has a field explicitly specifying the next desired state, as well as bits specifying possible modifications of that state. If specified, external signals are logically OR'd into the desired state number. In the prototype evaluator these external signals are: (1) the type bits from the E bus; (2) a bit which is 1 iff the E bus type field is zero and a bit which is 1 iff the E bus address is zero. In the storage manager these signals are: (1) the four control bits from the evaluator controller; (2) a bit which is 1 iff the G bus address is zero. This is the way in which dispatching is achieved.

Once this new state is computed, it is passed through a three-way selector before entering the state register. The other two inputs to the selector are the current state and the data lines from the external memory system. In this way the selector control can "freeze" a controller in its current state by recirculating it, or jam an externally supplied state into the state register (both useful for debugging operations). The "freeze" mechanism is used by the storage manager to suspend the evaluator until it is ready to process the next request. In the same way, the external memory can suspend the storage manager by asserting the external FREEZE signal, thereby causing a "wait state".

(The FREEZE signal is provided as a separate control because the dynamic logic techniques usual in NMOS were used; if one stopped the processor simply by stopping the clock, the register contents would dissipate. The clocks must keep cycling in order to "refresh" the registers. The state recirculation control allows the machine to be logically stopped despite the fact that data is still circulating internally. We discovered that this technique imposed constraints on other parts of the design: the incrementation logic is the best example. It was originally intended to design an incrementing counter register, which would increment its contents in place during a clock cycle in which an "increment" signal was asserted. If this had been done, however, and the processor were frozen during an instruction which asserted this signal, the counter would continue to count while the processor was stopped! This could have been patched by having the FREEZE signal override the increment signal, but it was deemed simpler to adopt a design strategy in which nothing at the microcode level called for any data to be read, modified, and stored back into the same place. Thus in the actual design one must read data through modification logic and then onto the bus, to be stored in a different register; then if this operation is repeated many times because of the FREEZE signal it makes no difference.)

Each state-machine controller consists of a read-only memory (implemented as a programmed-logic-array), two half-registers (clocked inverters, one at each input and one at each output), and some random logic (e.g., for computing the next state). The controllers are driven by externally supplied two-phase non-overlapping clock signals; on phase 1 the registers are clocked and the next state is computed, and on phase 2 the next-state signals appear and are latched.

All of the signals from the two controllers (62 = 34+28 in the prototype) are multiplexed onto twelve probe lines by six unary probe-control signals. (These signals are derived from three binary-encoded off-chip signals.) When a probe-control signal is asserted, the memory output pads (11 data pads plus the ADR signal in the prototype) are disconnected from the G bus and connected to the twelve probe lines. In this way the chip can be frozen and then all controller outputs verified (by cycling the probe-control signals through all six states). Also recall that the controller states can be jammed into the state registers from the memory input pads. This allows

the controller microcode to be tested completely without depending on the registers and busses working.

Figure 5 shows the physical layout of the prototype chip. The two controllers are side by side, with the evaluator on the left and the storage manager on the right. Above each controller is the next-state logic and probe multiplexor for that controller. Above those are the register arrays, with the busses running horizontally through them. The bus connections are in the center. The input pads are on the left edge, and the output pads on the right edge. The input pads are bussed through the evaluator's register array parallel to the E bus lines, so that they can connect to the G bus. (Unfortunately, there was no time to design tri-state pads for this project.)

Discussion

A perhaps mildly astonishing feature of this computer is that it contains no arithmetic-logic unit (ALU). More precisely, it does have arithmetic and logical capabilities, but the arithmetic units can only add 1, and the logical units can only test for zero. (Logicians know that this suffices to build a "three-counter machine", which is known to be as universal (and as convenient!) as a Turing Machine [Minsky 1967]. However, our LISP architecture is also universal, and considerably more convenient.)

LISP itself is so simple that the interpreter needs no arithmetic to run interesting programs (such as computing symbolic derivatives and integrals, or pattern matching). All the LISP interpreter has to do is shuffle pointers to and from memory, and occasionally dispatch on the type of a pointer. The incrementation logic is included on the chip for two reasons. In the evaluator it is used for counting down a list when looking up lexical variables in the environment; this is not really necessary, for there are alternative environment representation strategies. In the storage manager incrementation is necessary (and, in the prototype, sufficient) for imposing a total ordering on the external memory, so as to be able to enumerate all possible addresses. The only reason for adding 1 is to get to the next memory address. (One might note that the arithmetic properties of general two-argument addition are not exploited here. Any bijective mapping from the set of external memory addresses onto itself (i.e., a permutation function) would work just fine (but the permutation should contain only one cycle if memory is not to be wasted!). For example, subtracting 1 instead of adding, or Gray-code incrementation, would do.)

This is not to say that real LISP programs never use arithmetic. It is just that the LISP interpreter itself does not require binary arithmetic of the usual sort. (It does, however, require CONS, CAR, and CDR, which in a formal sense indeed form a kind of "number system" [Levin 1974], where CONS corresponds to "add 1" and both CAR and CDR to "subtract 1". In this view, the purpose of the storage manager is to interface between two kinds of arithmetic, namely "LISP arithmetic" and Peano arithmetic. This suggests one way to perform variable lookups without indexing or even arithmetic counting: a variable reference would be encoded as a list whose length indicates how far back to look in the environment. The lookup procedure would cdr the reference list and the environment together; when the one ran out, the car of the other would be the desired value.)

This architecture is intended to use devices which are addressed as memory, in the same manner used by the PDP-11, for example. We envision having a set of devices on the external memory bus which do arithmetic. One would then write operands into specific "memory locations" and then read arithmetic results from others. Such devices could be very complex processors in themselves, such as specialized array or string processors. In this way the LISP computer could serve as a convenient controller for other processors,

for one thing LISP does well is to provide recursive control and environment handling without much prejudice (or expertise!) as to the data being operated upon.

Expanding on this idea, one could arrange for additional signals to the external memory system from the storage manager, such as "this data item is needed (or not needed)", which would enable external processors to do their own storage management cooperatively with the LISP processor. One might imagine, for example, an APL machine which provided tremendous array processing power, controlled by a LISP interpreter specifying which operations to perform. The APL machine could manage its own array storage, using a relatively simple storage manager cued by "mark" signals from the LISP storage manager.

The possibility of additional processors aside, this architecture exhibits an interesting layered approach to machine design. One can draw boundaries at various places such that everything above the boundary is a processor which treats everything below the boundary as a memory system with certain operations. If the boundary is drawn between the evaluator and the storage manager, then everything below the boundary together constitutes a list-structure memory system. If it is drawn between the storage manager and the external memory, then everything below the boundary is the external memory. Supposing the external memory to be a cached virtual memory system, then we could draw boundaries between the cache and main memory, or between main memory and disks, and the same observation would hold. At the other end of the scale, a complex data base management system could be written in LISP, and then the entire LISP chip (plus some software, perhaps in an external ROM) would constitute a memory system for a data base query language interpreter. In this manner we have a layered series of processors, each of which provides a more sophisticated memory system to the processor above it in terms of the less sophisticated memory system below it.

Another way to say this is that we have a hierarchy of data abstractions, each implemented in terms of a more primitive one. Thus the storage manager makes a finite, linear memory look "infinite" and tree-structured. A cache system makes a large, slow memory plus a small, fast memory look like a large, fast memory.

Yet another way to view this is as a hierarchy of interpreters running in virtual machines. Each layer implements a virtual machine within which the next processor up operates.

It is important to note that we may choose any boundary and then build everything below it in hardware and everything above it in software. Our LISP system is actually quite similar to those before it, except that we have pulled the hardware boundary much higher. One can also put different layers on different chips (as with the LISP chip and its memory). We choose to put the evaluator and the storage manager on the same chip only because (a) they

fit, and (b) in the planned full-scale version, the storage manager would need too many pins as a separate chip.

Each of the layers in this architecture has much the same organization: it is divided into a controller ("state machine") and a data base ("registers"). There is a reason for this. Each layer implements a memory system, and so has state; this state is contained in the data base (which may be simply a small set of references into the next memory system down). Each layer also accepts commands from the layer above it, and transforms them into commands for the layer below it; this is the task of the controller.

We have already mentioned some of the analogies between a LISP-based processor and a traditional processor. Corresponding to indexing there is component selection; corresponding to a linearly advancing program counter there is recursive tree-walk of expressions. Another analogy we might draw is to view the instruction set as consisting of variable-length instructions (whose pieces are joined by pointers rather than being arranged in sequential memory locations). Each instruction (variable reference, call to CONS, call to user function, etc.) takes a number of operands. We may loosely say that there are three addressing modes in this architecture: immediate data, as with a quoted constant; lookup of a named value, as in a variable reference; and recursive evaluation. In the last case, merely referring to an operand automatically calls for the execution of an entire routine to compute it!

Conclusions

We have presented a general design for and a specific example of a new class of hardware processors. This model is "classical" in that it exhibits the stored-program, program-as-data idea, as well as the processor/memory dichotomy which leads to the so-called "von Neumann bottleneck" [Backus 1978]. It differs from the usual stored-program computer in organizing its memory differently, and in using an instruction set based on this memory organization. Where the usual computer treats memory as a linear vector and executes a linear instruction stream, the architecture we present treats memory as linked records, and executes a tree-shaped program by recursive expression evaluation.

The processor described here is not to be confused with the "LISP Machine" designed and built at MIT by Greenblatt and Knight [Greenblatt 1974] [Knight 1974] [LISP Machine 1977] [Weinreb 1978]. The current generation of LISP Machine is built of standard Schottky TTL, and its hardware is organized as a very general-purpose microprogrammed processor of the traditional kind. It has a powerful arithmetic-logic unit and a large writable control store. Almost none of the hardware is specifically designed to handle LISP code; it is the microcode which customizes it for LISP. Finally, the LISP Machine executes a compiled order code which is of the linearly-advancing-PC type;

the instruction set deals with a powerful stack machine. Thus the LISP Machine may be thought of as a hybrid architecture that takes advantage of linear vector storage organization and stack organization as well as linked-list organization. In contrast, the class of processors we present here is organized purely around linked records, especially in that the instruction set is embedded in that organization. The LISP Machine is a well-engineered machine for general-purpose production use, and so uses a variety of storage-management techniques as appropriate. The processor described here is instead intended as an illustration of the abstracted essence of a single technique, with as little additional context or irrelevant detail as possible.

We have designed and fabricated SCHEME-78, a prototype LISP-based processor. The actual hardware design and layout was done by one of the authors (Steele) as a term project for a course on VLSI given at MIT by Lynn Conway in Fall 1978. The prototype processor has a small but complete expression evaluator, and an incomplete storage manager (everything but the garbage collector). It also has a fatal artwork error (akin to a typographical error — three small pieces of metal were accidentally omitted), which prevented any significant testing of the fabricated parts. Further information on this specific design, including a complete logic diagram, may be found in [Steele 1979b]; see also [Steele 1979a].

In 1979 we (with Holloway and Bell [Holloway 1980]) designed and fabricated a full-scale VLSI processor (known as SCHEME-79) having a complete garbage collector, with more built-in primitive operations, and a more complex storage representation. This processor has 32-bit words, divided into 7 type bits, 24 address bits, and 1 bit reserved for the garbage collector. It has more powerful arithmetic and logical capabilities: it can subtract 1 as well as add 1, and can test for equality rather than merely test for zero. Testing of this chip is not complete; however, it has successfully executed a few non-trivial programs and survived several garbage collections. Preliminary indications are that it interprets pure LISP code at approximately the same speed as a DEC PDP-10 model KA10 processor.

We intend to design and fabricate further versions of this architecture. The SCHEME-79 chip was designed with no regard for speed, and we estimate that careful attention to electrical considerations can improve performance severalfold. Algorithmic and data structure improvements (such as "CDR-coding" [Hansen 1969] [Greenblatt 1974], "dreaming" [Steele 1979c], and "phantom stacks" (Richard Stallman, private communication)) will probably gain another factor of four. Thus we estimate a potential overall improvement of 10 to 20, making this interpretive architecture competitive with compiled code for more traditional computers.

A final philosophical thought: it may be worth considering kinds of "stuff" other than vectors and linked records to use for representing data. For example, in LISP we generally organize the records only into trees rather

than general graphs. Other storage organizations should also be explored. The crucial idea, however, is that the instruction set should then be fit into the new storage structure in some natural and interesting way, thereby representing programs in terms of the data structures. Continuing the one example, we might look for an evaluation mechanism on general graphs rather than on trees, or on whatever other storage structure we choose. Finally, the instruction set, besides being represented in terms of the data structures, must include means for manipulating those structures. Just as the usual computer has ADD and AND; just as the LISP architecture presented here must supply CAR, CDR, and CONS; so a graph architecture must provide graph manipulation primitives, etc. Following this paradigm we may discover yet other interesting architectures and interpretation mechanisms.

Acknowledgements

We are very grateful to Lynn Conway for coming to MIT, teaching the techniques for NMOS design, and providing an opportunity for us to try our ideas as part of the course project chip. The text used for the course was written by Carver Mead and Lynn Conway (a 1978 draft of [Mead 1980]). Additional material was written by Bob Hon and Carlo Sequin [Hon 1978]. It should be mentioned that the course enabled a large number of students to try interesting and imaginative LSI designs as part of the project chip. This paper describes only one project of the set, but many of these student projects may have useful application in the future.

Paul Penfield and Jon Allen made all this possible by organizing the LSI design project at MIT and arranging for Charles Botchek and Lynn Conway to teach.

Charles Botchek provided our first introduction to the subject and started our wheels spinning.

The course and project chip were executed with the cooperation, generosity, and active help of the Xerox Palo Alto Research Center (PARC), which provided software and design support; Micromask, which generated the masks; and Hewlett-Packard, which fabricated the wafers at their Deer Creek facility.

Dick Lyon and Alan Bell of Xerox PARC performed plots of the projects and assembled the projects into the final mask specifications. They were of particular direct aid to Steele in debugging his project.

Glen Miranker and William Henke maintained the plotting software used at MIT to produce intermediate plots of student projects during the design cycle, and were helpful in making modifications to the software to accommodate this project.

Dmitri Antoniadis of MIT packaged and bonded the chips in 40-pin DIPs. Prof. Antoniadis was also a source of good advice concerning device physics and fabrication.

Peter Deutsch and Fernando Corbato were kind enough to hand-carry project plots from California to Boston to help meet the project deadline.

Tom Knight and Jack Holloway provided useful suggestions and sound engineering advice, as usual. (In particular, Knight helped Steele to design a smaller pad to reduce the area of the project, and Holloway suggested the probe multiplexor technique for testing internal signals.)

We thank the referees for their comments and encouragement.

Guy Steele's graduate studies during 1978-1980 are supported by a Fannie and John Hertz Fellowship. In the spring of 1978 they were supported by a National Science Foundation Graduate Fellowship.

References

(Reference [Berkeley 1964] is not referred to in the text.)

[Backus 1978]

Backus, John. Can programming be liberated from the von Neumann style? A functional style and its algebra of programs. Comm. ACM 21, 8 (August 1978), 613-641.

[Baker 1978]

Baker, Henry B., Jr. List processing in real time on a serial computer. Comm. ACM 21, 4 (April 1978), 280-294.

[Berkeley 1964]

Berkeley, Edmund C., and Bobrow, Daniel G. (Eds.) The Programming Language LISP: Its Operation and Applications. Information International, Inc. (Cambridge, 1964).

[Bobrow 1973]

Bobrow, Daniel G. and Wegbreit, Ben. A model and stack implementation of multiple environments. Comm. ACM 16, 10 (October 1973) pp. 591-603.

[Church 1941]

Church, Alonzo. The Calculi of Lambda Conversion. Annals of Mathematics Studies Number 6. Princeton University Press (Princeton, 1941). Reprinted by Klaus Reprint Corp. (New York, 1965).

[Conrad 1974]

Conrad, William R. A compactifying garbage collector for ECL's non-homogeneous heap. Technical Report 2-74. Center for Research in Computing Technology, Harvard U. (Cambridge, February 1974).

[Deutsch 1976]

Deutsch, L. Peter, and Bobrow, Daniel G. An efficient, incremental, automatic garbage collector. Comm. ACM 19, 9 (September 1976), 522-526.

[Galley 1975]

Galley, S.W. and Pfister, Greg. The MDL Language. Programming Technology Division Document SYS.11.01. Project MAC, MIT (Cambridge, November 1975).

[Greenblatt 1974]

Greenblatt, Richard. The LISP Machine. AI Working Paper 79. MIT AI Lab (Cambridge, November 1974).

[Gries 1977]

Gries, David. An exercise in proving parallel programs correct. Comm. ACM 20, 12 (December 1977), 921-930.

[Hansen 1969]

Hansen, Wilfred J. Compact list representation: definition, garbage collection, and system implementation. Comm. ACM 12, 9 (September 1969), 499-507.

[Hart 1964]

Hart, Timothy P., and Evans, Thomas G. Notes on implementing LISP for the M-460 computer. In [Berkeley 1964], pp. 191-203.

[Hewitt 1977]

Hewitt, Carl. Viewing control structures as patterns of passing messages. AI Journal 8, 3 (June 1977), 323-364.

[Holloway 1980]

Holloway, Jack; Steele, Guy Lewis Jr.; Sussman, Gerald Jay; and Bell, Alan. The SCHEME-79 chip. AI Memo 559. MIT Artificial Intelligence Laboratory (Cambridge, January 1980).

[Hon 1978]

Hon, Robert, and Sequin, Carlo. A Guide to LSI Implementation. Xerox PARC (Palo Alto, September 1978).

[Knight 1974]

Knight, Tom. The CONS microprocessor. AI Working Paper 80. MIT AI Lab (Cambridge, November 1974).

[Knuth 1968]

Knuth, Donald E. The Art of Computer Programming, Volume 1: Fundamental Algorithms. Addison-Wesley (Reading, Mass., 1968).

[Levin 1974]

Levin, Michael. Mathematical logic for computer scientists. TR-131. MIT Project MAC (Cambridge, June 1974).

[LISP Machine 1977]

The LISP Machine Group: Bawden, Alan; Greenblatt, Richard; Holloway, Jack; Knight, Thomas; Moon, David; and Weinreb, Daniel. LISP Machine progress report. AI Memo 444. MIT AI Lab (Cambridge, August 1977).

[McCarthy 1962]

McCarthy, John, et al. LISP 1.5 Programmer's Manual. MIT Press (Cambridge, 1962).

[McDermott 1974]

McDermott, Drew V. and Sussman, Gerald Jay. The CONNIVER reference manual. AI Memo 295a. MIT AI Lab (Cambridge, January 1974).

[Mead 1980]

Mead, Carver, and Conway, Lynn. Introduction to VLSI Systems. Addison-Wesley (Reading, Massachusetts, 1980).

[Minsky 1963]

Minsky, Marvin L. A LISP garbage collector using serial secondary storage. AI Memo 58 (revised). MIT AI Lab (Cambridge, December 1963).

[Minsky 1967]

Minsky, Marvin L. Computation: Finite and Infinite Machines. Prentice-Hall (Englewood Cliffs, New Jersey, 1967).

[Moon 1974]

Moon, David A. MacLISP reference manual, revision 0. Project MAC, MIT (Cambridge, April 1974).

[Morris 1978]

Morris, F. Lockwood. A time- and space-efficient garbage compaction algorithm. Comm. ACM 21, 8 (August 1978), 662-665.

[Moses 1970]

Moses, Joel. The function of FUNCTION in LISP. AI Memo 199. MIT AI Lab (Cambridge, June 1970).

[Reynolds 1972]

Reynolds, John C. Definitional interpreters for higher order programming languages. Proc. ACM National Conference (Boston, 1972), 717-740.

[Saunders 1964]

Saunders, Robert A. The LISP system for the Q-32 computer. In [Berkeley 1964], 220-231.

[Schorr 1967]

Schorr, H., and Waite, W. M. An efficient machine-independent procedure for garbage collection in various list structures. Comm. ACM 10, 8 (August 1967), 501-506.

[Steele 1977a]

Steele, Guy Lewis Jr. Compiler Optimization Based on Viewing LAMBDA as Rename plus Goto. S.M. thesis. MIT (Cambridge, May 1977).

[Steele 1977b]

Steele, Guy Lewis Jr. Debunking the "expensive procedure call" myth. Proc. ACM National Conference (Seattle, October 1977), 153-162. Revised as AI Memo 443, MIT AI Lab (Cambridge, October 1977).

[Steele 1978a]

Steele, Guy Lewis Jr., and Sussman, Gerald Jay. The revised report on SCHEME: a dialect of LISP. AI Memo 452. MIT AI Lab (Cambridge, January 1978).

[Steele 1978b]

Steele, Guy Lewis Jr., and Sussman, Gerald Jay. The art of the interpreter; or, the modularity complex (parts zero, one, and two). AI Memo 453. MIT AI Lab (Cambridge, January 1978).

[Steele 1979a]

Steele, Guy Lewis Jr., and Sussman, Gerald Jay. Storage management in a LISP-based processor. Proc. Caltech Conference on Very Large Scale Integration (Pasadena, January 1979).

[Steele 1979b]

Steele, Guy Lewis Jr., and Sussman, Gerald Jay. Design of LISP-based processors; or, SCHEME: a dielectric LISP; or, finite memories considered harmful; or, LAMBDA: the ultimate opcode. AI Memo 514. MIT AI Lab (Cambridge, March 1979).

[Steele 1979c]

Steele, Guy Lewis Jr., and Sussman, Gerald Jay. The dream of a lifetime: a lazy scoping mechanism. AI Memo 527. MIT AI Lab (Cambridge, November 1979).

[Sussman 1975]

Sussman, Gerald Jay, and Steele, Guy Lewis Jr. SCHEME: an interpreter for extended lambda calculus. AI Memo 349. MIT AI Lab (Cambridge, December 1975).

[Wand 1977]

Wand, Mitchell. Continuation-based program transformation strategies. Technical Report 61. Computer Science Department, Indiana University (Bloomington, March 1977). Also in J. ACM 27, 1 (January 1980), 164-180.

[Wegbreit 1974]

Wegbreit, Ben, et al. ECL programmer's manual. Center for Research in Computing Technology, Harvard U. (Cambridge, December 1974).

[Weinreb 1978]

Weinreb, Daniel, and Moon, David. LISP Machine manual (preliminary version). MIT AI Lab (Cambridge, November 1978).